**Program -2: Finance Assistant: Budget Advisor with Tool Use**

Design a financial assistant that answers natural language queries about budgeting and interest calculations by invoking calculator tools or financial APIs (e.g., currency rates).

This manual documents the Langflow workflow implemented for the program. The workflow uses OpenRouter as the connected language model source, an Agent as the control component, the built-in Calculator for arithmetic, and a custom Currency Converter Tool for live exchange-rate support. The document focuses on the actual flow of information between components and explains how the system processed the sample inputs successfully shown in Playground.

## 1. Aim and Finalized Architecture

Aim: To build a finance assistant in Langflow that can receive natural language queries, interpret them using a connected LLM, invoke Calculator for arithmetic queries such as budgeting and simple interest, optionally invoke a live currency-conversion tool for exchange-rate queries, and return the final answer to the user through the chat interface.

### 1.1 Component chain

- Chat Input receives the user query from Playground and injects it into the flow.
- OpenRouter provides the connected language model to the Agent.
- The Agent interprets the query, decides whether a tool is required, and prepares the final response.
- Calculator is available to the Agent for arithmetic operations.
- Currency Converter Tool is available to the Agent for live currency-conversion requests.

• Chat Output displays the final response back in Playground.

### 1.2 Overall block diagram

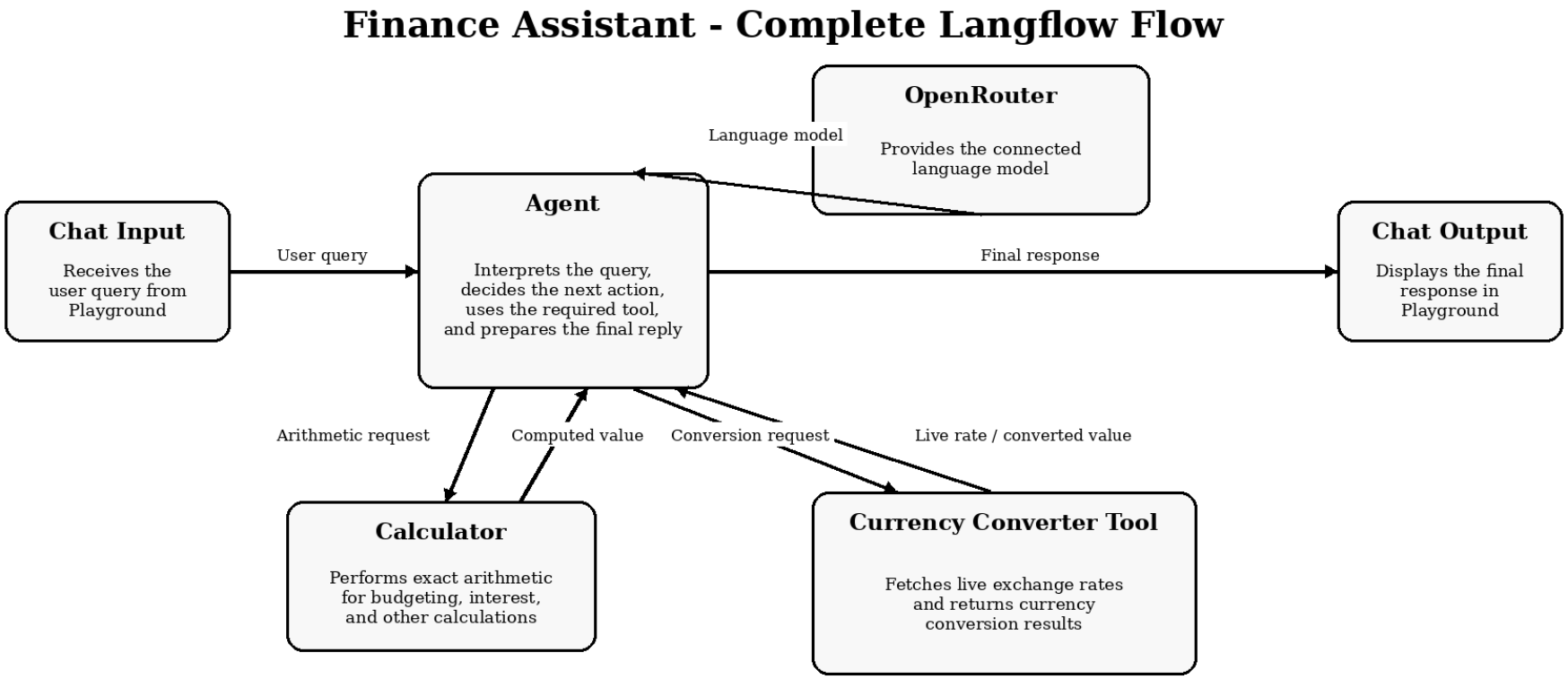


*Figure 1. Overall Langflow architecture and direction of information flow.*

## 2. Purpose of Each Component and What Flows Through It

| Component | What it receives | What it sends | Purpose in the workflow |
|---|---|---|---|
| Chat Input | User message typed in Playground | Message object to Agent | Acts as the entry point of the workflow. |
| OpenRouter | API key, chosen model, and internal prompt context | Language Model connection to Agent | Supplies the LLM that the Agent uses for reasoning and response generation. |
| Agent | User message, Agent instructions, connected tools, connected language model | Final response to Chat Output; tool requests to Calculator or Currency Converter Tool when required | Acts as the controller of the entire system. |
| Calculator | Arithmetic expression from Agent | Computed value back to Agent | Provides exact arithmetic for budgeting and interest calculations. |
| Currency Converter Tool | Strict input string in the form amount FROM TO | Converted currency value back to Agent | Provides live currency conversion using an online exchange-rate API. |
| Chat Output | Final response from Agent | Visible reply in Playground | Displays the final answer to the user. |

*Reading the table from left to right gives the runtime meaning of each component: what enters the component, what leaves it, and why that component exists in the design.*

## 3. Runtime Flow of the Complete System

The workflow starts when the user types a message in Playground and clicks Send. That message first reaches Chat Input. Chat Input passes it to the Agent as the input message for the current run. The Agent then uses the connected OpenRouter language model to understand the request, applies the Agent Instructions, checks whether a tool is required, invokes the appropriate tool if needed, and finally sends the response to Chat Output. Chat Output displays the reply in Playground.

### 3.1 Step-by-step execution path

1. The user enters a query in Playground.

2. Playground forwards the query into Chat Input.

3. Chat Input passes the message to the Agent.

4. The Agent uses the OpenRouter-connected language model to interpret the message.

5. The Agent decides whether the query is best handled directly by the LLM or by invoking one of the connected tools.

6. For arithmetic or finance calculations, the Agent can invoke Calculator.

7. For live exchange-rate requests, the Agent can invoke Currency Converter Tool.

8. The selected tool returns its result to the Agent.

9. The Agent combines the result with a clear natural-language explanation.

10. Chat Output displays the final response back in Playground.

## 4. Implemented Workflow in Langflow

Figure 2 shows the finalized workflow built in Langflow. The key connections visible in the workflow are: OpenRouter to Agent as the language model source, Chat Input to Agent as the user-message path, Calculator to Agent Tools, Currency Converter Tool to Agent Tools, and Agent to Chat Output as the final response path.

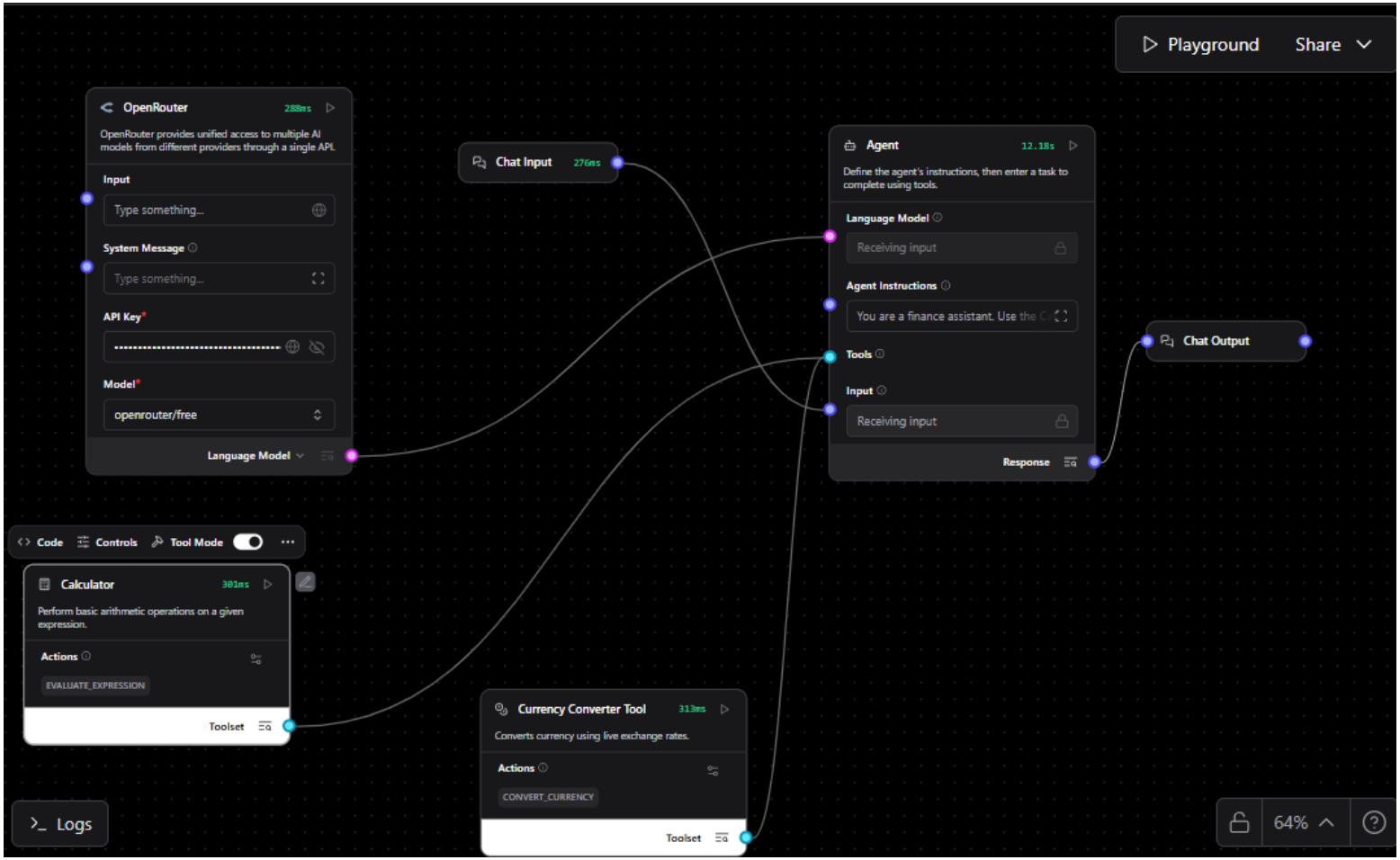


*Figure 2. Implemented Langflow workflow used for the finance assistant.*

## 5. Configuration Notes Used in This Implementation

### 5.1 OpenRouter setup

The OpenRouter component was configured as the language-model provider for the Agent. In this implementation, the API key was generated through the OpenRouter platform at https://openrouter.ai/ and then placed in the OpenRouter component. The model selected for the working setup was openrouter/free. The OpenRouter component output was configured as Language Model and connected to the Agent Language Model input.

### 5.2 Agent Instructions

The Agent Instructions define how the Agent should reason over the incoming query and when it should use tools. A concise instruction strategy is recommended: use Calculator for arithmetic and finance calculations, use Currency Converter Tool for live currency conversion, and return clear answers.

```
You are a finance assistant.
Use the Calculator tool for arithmetic, budgeting, and interest calculations.
Use the Currency Converter Tool for live currency conversion.
When calling the Currency Converter Tool, always pass the input in the form: amount FROM TO. Example:
100 USD INR.
Give clear and concise answers. If a value is missing, ask for it.
```

### 5.3 Currency Converter Tool code

The following custom component code is used for the live currency-conversion extension.

```
import requests
from langflow.custom import Component
from langflow.io import MessageTextInput, Output
from langflow.schema import Data


class CurrencyConverterTool(Component):
    display_name = "Currency Converter Tool"
    description = "Converts currency using live exchange rates."
    icon = "coins"
    name = "CurrencyConverterTool"

    inputs = [
        MessageTextInput(
            name="query",
            display_name="Conversion Request",
            info="Use format: 100 USD INR",
            tool_mode=True,
        ),
    ]
```

```
    outputs = [
        Output(display_name="Result", name="result", method="convert_currency"),
    ]

    def convert_currency(self) -> Data:
        try:
            parts = self.query.upper().split()

            amount = parts[0]
            base = parts[1]
            target = parts[2]

            url = f"https://api.frankfurter.dev/v1/latest?base={base}&symbols={target}"
            response = requests.get(url)
            data = response.json()

            rate = data["rates"][target]
            converted = float(amount) * rate

            result = f"{amount} {base} = {converted:.2f} {target}"
            return Data(value={"result": result})

        except:
            return Data(value={"result": "Use format: 100 USD INR"})
```

## 6. Case Study Analysis of the Two Playground Inputs

Each case study explains the observed input, the intended processing path through the workflow, the decision made by the Agent, and the final output that appeared in Playground.

### 6.1 Case Study - Simple Interest Query

Input used in Playground: Calculate simple interest for 20000 at 8 percent for 2 years


*Figure 3. Playground result for the simple-interest query.*

| Stage | Observed and interpreted flow |
|---|---|
| User input | The query is entered in Playground and transferred to Chat Input. |
| Message transfer | Chat Input sends the message to the Agent as the active request. |
| Agent understanding | The Agent uses the connected OpenRouter language model to identify that the user is asking for a simple-interest calculation and that the relevant values are Principal = 20000, Rate = 8, and Time = 2 years. |
| Tool decision | The designed tool path for this type of query is Calculator because the task is a finance calculation based on a formula. |
| Arithmetic step | The simple-interest expression is SI = (P × R × T) / 100 = (20000 × 8 × 2) / 100 = 3200. |
| Final response construction | The Agent formats the result in natural language and sends the final answer to Chat Output. |
| Displayed output | The Playground shows the answer: The simple interest is 3200. |

### 6.2 Case Study - Direct Arithmetic Query

Input used in Playground: what is 2+3

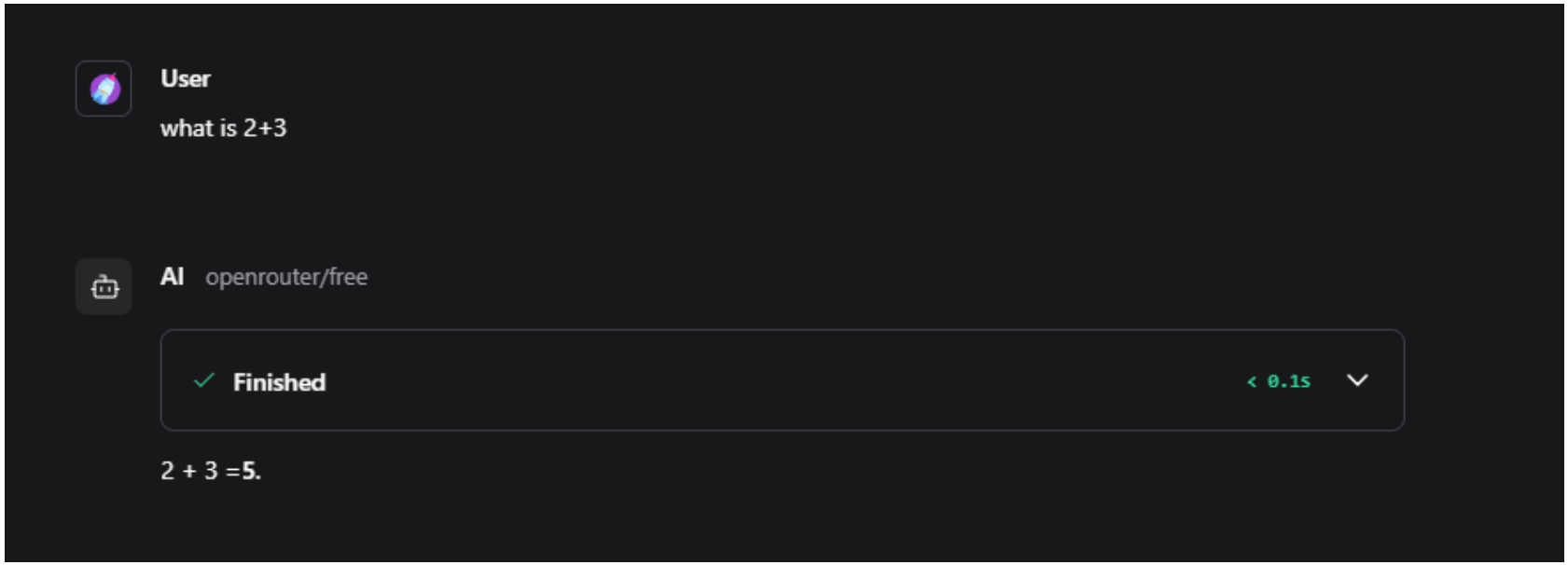


*Figure 4. Playground result for the direct arithmetic query.*

| Stage | Observed and interpreted flow |
|---|---|
| User input | The expression is entered in Playground and transferred to Chat Input. |
| Message transfer | Chat Input passes the message to the Agent. |
| Agent understanding | The Agent uses the connected language model to interpret the request as a |

| | |
|---|---|
| | simple arithmetic question. |
| Decision point | The architecture provides Calculator as the arithmetic tool. However, for very small expressions such as 2 + 3, an LLM-based agent may answer directly without issuing a tool call. |
| Final response construction | The Agent prepares the short response 2 + 3 = 5 and sends it to Chat Output. |
| Displayed output | The Playground shows the answer: 2 + 3 = 5. |

## 7. What the Workflow Demonstrates

- The OpenRouter components role is to provide the language model used by the Agent.
- The Agent is the central decision-making component. It receives the message, reasons about the request, selects tools when required, and builds the final answer.
- Calculator and Currency Converter Tool are not independent front-end components. They act as tools only when the Agent calls them.
- Chat Input and Chat Output are required to make the workflow operate properly through the Playground chat interface.
- The workflow separates reasoning from specialized operations: reasoning through the connected LLM, arithmetic through Calculator, and live exchange-rate retrieval through the currency tool.

## 8. Result and Conclusion

The finalized workflow successfully implements the finance assistant required in Program - 2. The Langflow design clearly separates message entry, reasoning, tool invocation, and final response display. The working screenshots confirm that the workflow handles simple-interest and direct arithmetic queries correctly. The connected Currency Converter Tool further extends the same architecture to support live exchange-rate queries without changing the main control logic of the Agent.

The resulting system is a complete agentic workflow in which the Agent serves as the controller, OpenRouter provides the language model, Calculator supplies arithmetic capability, Currency Converter Tool supplies live exchange-rate capability, and the Playground conversation is managed through Chat Input and Chat Output.

## Appendix - Note on OpenRouter and Model Access

OpenRouter is a unified AI model access platform that allows users to connect to multiple AI models through a single API. Instead of using separate APIs for different model providers, OpenRouter provides one common interface through which various models can be accessed.

In the context of this lab manual, OpenRouter is useful because it simplifies the process of connecting an application or workflow to AI models. Once the user creates an API key from the OpenRouter platform, that key can be used to access supported models inside the workflow.

A key advantage of OpenRouter is that it supports both free and paid models. This means a learner can begin experimentation using free models and later move to paid models when higher performance, reliability, or advanced capabilities are required. This flexibility is helpful in educational and development environments, where initial testing can be done at low or no cost.

In simple terms:

- OpenRouter acts as a single gateway to access multiple AI models.
- It reduces the need to separately manage different provider APIs.
- It allows users to work with both free and paid models from one place.
- It is useful for experimentation, learning, prototyping, and application development.

For this work, the OpenRouter API key was generated using the official platform:

**OpenRouter Website: https://openrouter.ai/**

This platform can therefore be considered as the service layer through which the AI model is connected to the workflow used in the lab program.